MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 11
AĞAÇLAR

Dr. Öğr. Üyesi Ebru AYDOĞAN

Bucak Bilgisayar ve Bilişim Fakültesi

Ders İçeriği: Hafta 11

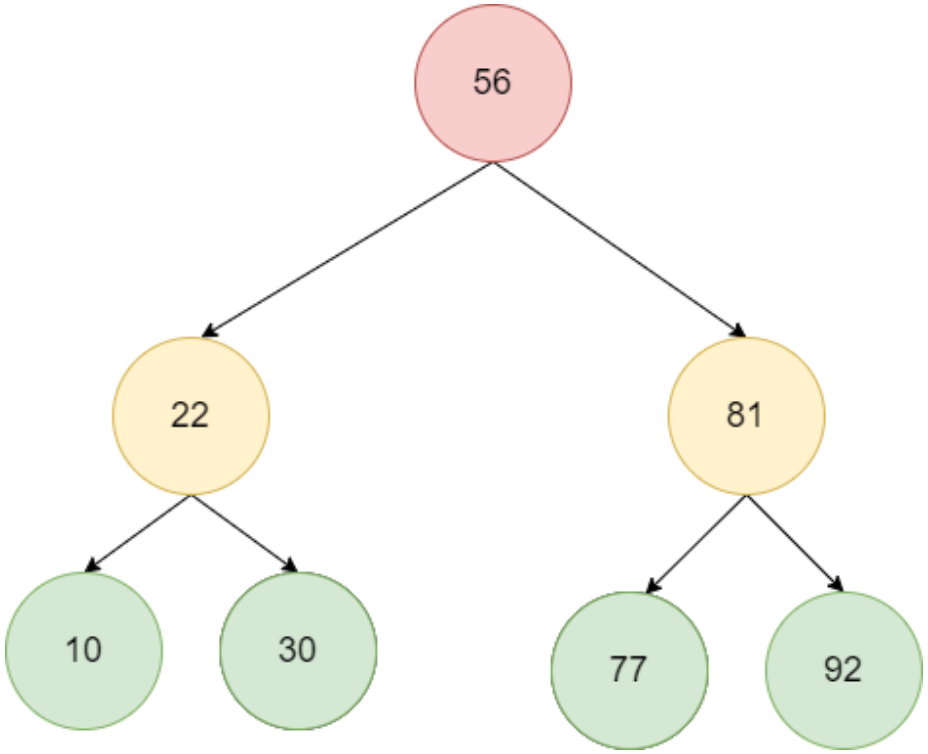
- Temel Kavramlar
- Ağaçlara İlişkin Tanımlar
- Ağaçların Bağlı Yapısı
- Ağaç İşlemleri
- Ağaç Türleri
- İkili Ağaçlar
- Heap Tree (Kümeleme Ağacı)
- Trie Ağacı/Sözlük Ağacı
- İkili Ağaçlarda Geçiş İşlemleri
- İfade Ağacı (Expression Tree)

Temel Kavramlar

- Ağaç, verilerin birbirine sanki bir ağaç yapısı oluşturuyormuş gibi sanal olarak bağlanmasıyla elde edilir.
- Ağaç yapısında veriler düğümlerde (node) tutulur.
- Ağaç, bir kök işaretçisi, sonlu sayıda düğümleri ve onları birbirine bağlayan dalları olan bir veri modelidir; aynı aile soyağacında olduğu gibi hiyerarşik bir yapısı vardır ve orada geçen birçok kavram buradaki ağaç veri modelinde de tanımlıdır.
- Her biri değişik bir uygulamaya doğal çözüm olan ikili ağaç, kodlama ağacı, sözlük ağacı, kümeleme ağacı gibi çeşitli ağaç şekilleri vardır; üstelik uygulamaya yönelik özel ağaç şekilleri de çıkarılabilir.

Temel Kavramlar

- Bağlı listeler, yığınlar ve kuyruklar doğrusal (lineer) veri yapılarıdır.
- Ağaçlar ise doğrusal olmayan belirli niteliklere sahip iki boyutlu veri yapılarıdır.

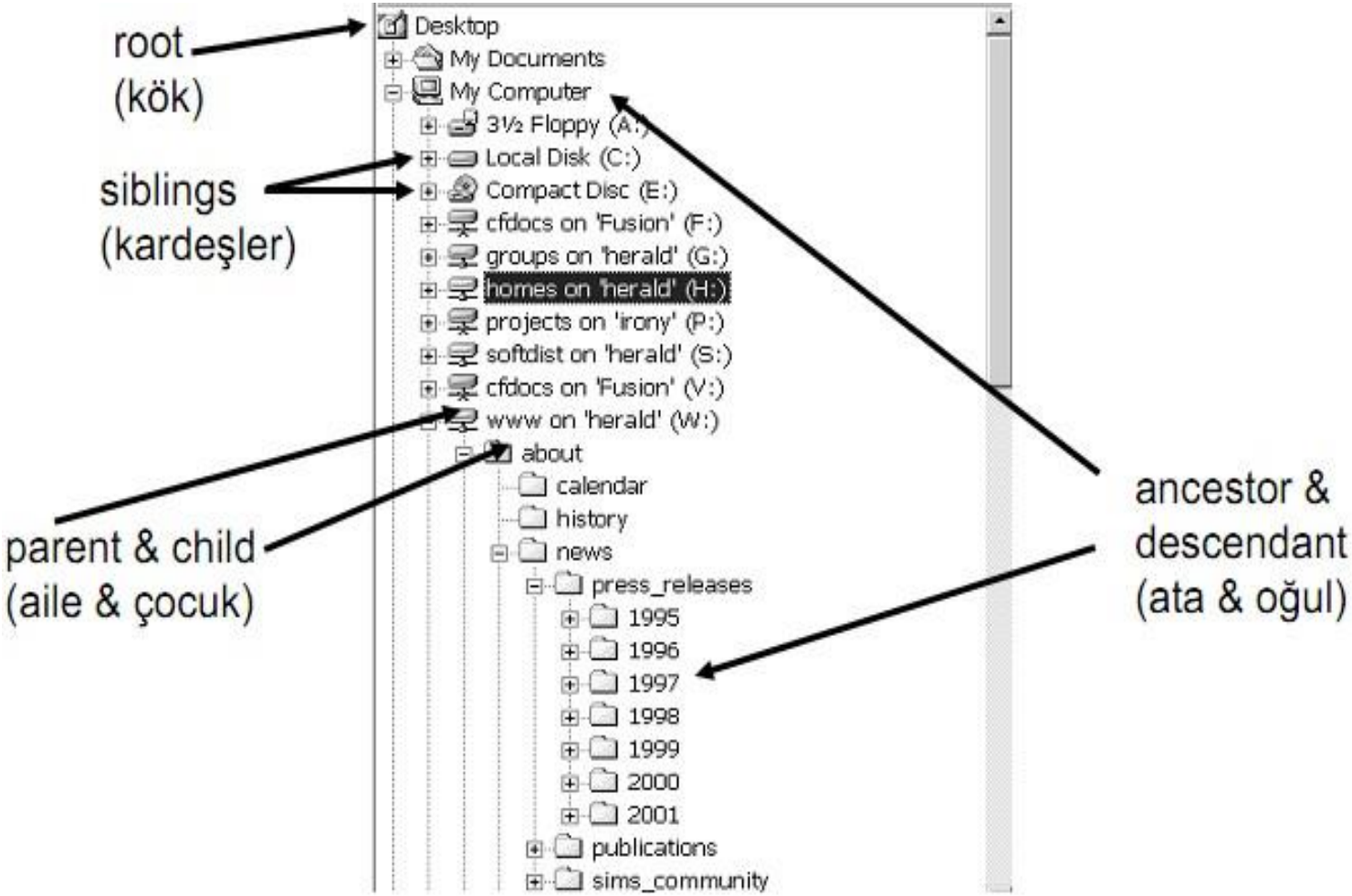


Temel Kavramlar

- Ağaçlar hiyerarşik ilişkileri göstermek için kullanılır.
- Veriler bir ağaç yapısına benzer şekilde (kök gövde yapraklar) tutulur.
- Her ağaç kök, düğümler ve yapraklardan oluşur.
- İkili ağaçlar (binary trees), en fazla iki alt düğüm içerir.
- Arama işlemi doğrusal listelere göre çok daha hızlı yapılır.
- Uygulamaları:
  - Organizasyon şeması
  - Dosya sistemleri
  - Programlama ortamları

Temel Kavramlar

- Dosya sistemleri için örnek bir ağaç yapısı



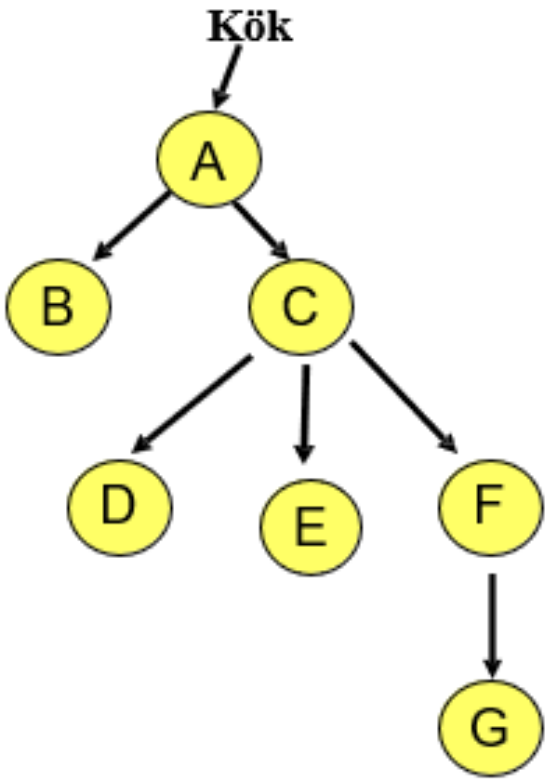
Ağaçlara İlişkin Tanımlar

- Düğüm (Node): Ağacın her bir elemanına düğüm adı verilir.
- Kök Düğüm (Root): Ağacın en üstteki başlangıç düğümüdür.
- Çocuk (Child): Bir düğüme doğrudan bağlı alt düğümlere denir.
- Kardeş Düğüm (Sibling): Aynı düğüme bağlı düğümlere kardeş düğüm denir.
- Aile/Ebeveyn (Parent): Düğümlerin doğrudan bağlı oldukları düğüm aile olarak adlandırılır; diğer bir deyişle aile, kardeşlerin bağlı olduğu düğümdür.
- Ata (Ancestor): Aile düğümünün daha üstünde kalan düğümlere ata denilir.
- Torun (Dedscendant): Bir düğümün çocuğuna bağlı olan düğümlere denir.

Ağaçlara İlişkin Tanımlar

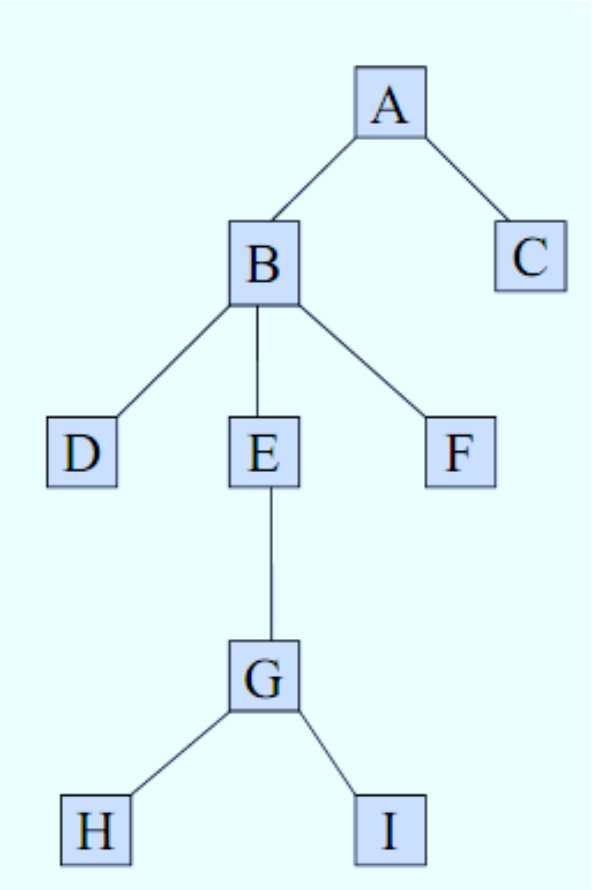
- Derece (Çocuk): Bir düğümden alt hiyerarşiye yapılan bağlantıların sayısı yani çocuk sayısıdır.
- Derinlik (Depth): Bir düğümün köke olan uzaklığı derinliktir. Kök düğümün derinliği 0'dır.
- Yaprak (Leaf): Ağacın en altında bulunan ve çocukları olmayan düğümlerdir.
- Yükseklik (Height): Bir düğümün kendi silsilesinden en uzak yaprak düğüme olan uzaklığıdır. Yaprak düğümlerin yüksekliği 0'dır.
- Yol (Path): Bir düğümden aşağıya doğru bir başka düğüme gidebilmek için üzerinden geçilmesi gereken düğümlerin listesidir.

Ağaçlara İlişkin Tanımlar



Tanım	Kök	B	D
Derece	2	0	0
Kardeş	1	2	3
Çocuk	2(B,C)	yok	yok
Aile	yok	Kök	C
Ata	yok	yok	Kök
Yol	A	A, B	A,C,D
Derinlik	0	1	2
Yükseklik	3	0	0

Ağaçlara İlişkin Tanımlar



Tanım	Değer
Düğüm sayısı:	9
Yükseklik:	4
Kök düğüm:	A
Yapraklar:	C, D, F, H, I
D’nin derinliği:	2
H'nin ataları:	E, B, A
B'nin torunları:	G, H, I
E'nin kardeşleri:	D, F
Sağ alt ağaç:	Yok
Sol alt ağaç:	B

Ağaçların Bağlı Yapısı

- Çocuk/Kardeş Gösterimi
  - Her düğümde hem veri hem de çocukların adresi tutulur.
  - Örnek ikili ağaç tanımı:

```
struct AgacDugumu {
  int veri;
  AgacDugumu *sol;   // ilk çocuk
  AgacDugumu *sag;  // kardeş
  };
```

Ağaç İşlemleri

- Genel Yöntemler:
  - integer size()
  - boolean isEmpty()
  - elements()
  - positions()
- Erişim yöntemleri:
  - root()
  - parent(p)
  - children(p)
- Sorgu (Query) Yöntemleri:
  - boolean isInternal(p)
  - boolean isExternal(p)
  - boolean isRoot(p)
- Güncelleme (Update) Yöntemi:
  - replace (p, o)
- Ek yöntemler de tanımlanabilir

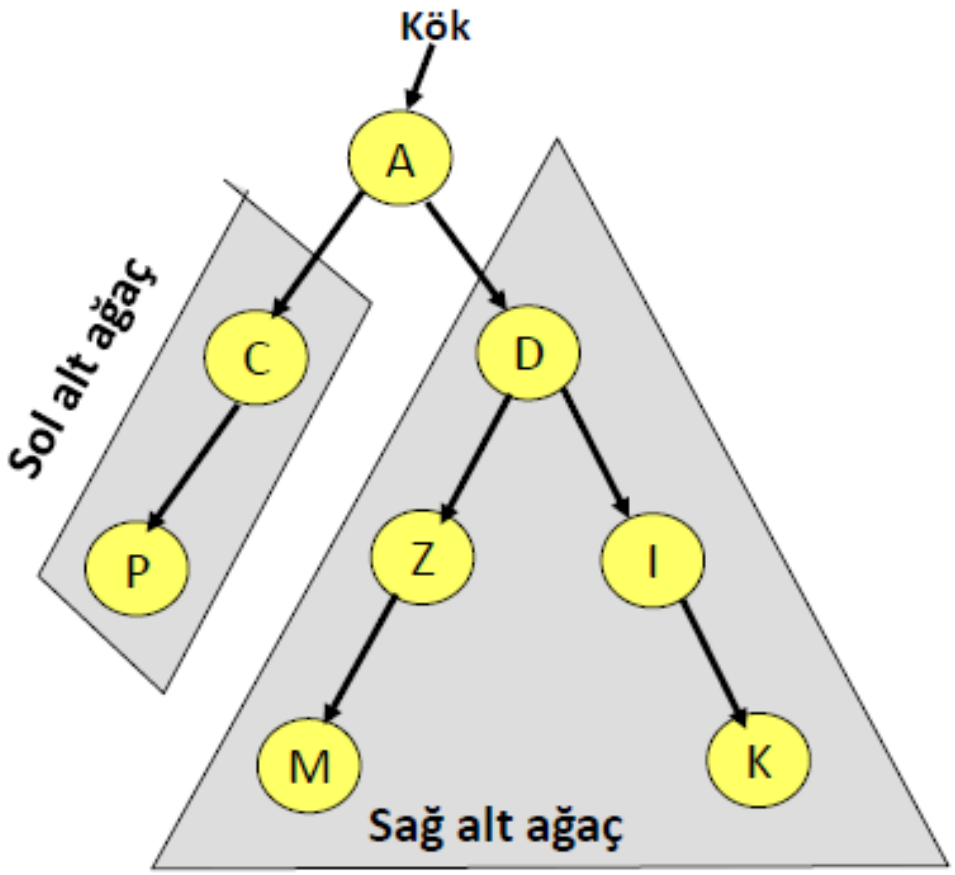
Ağaç Türleri

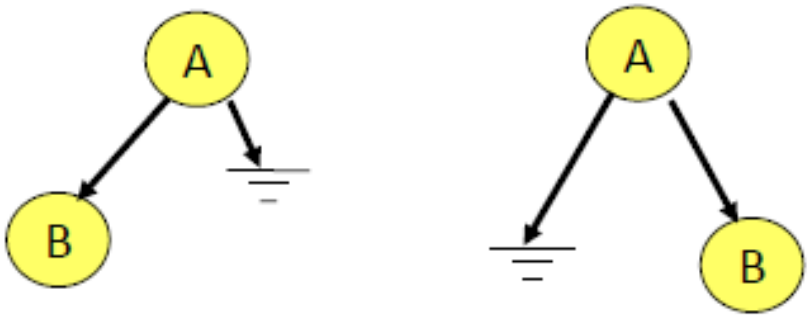
- İkili Arama Ağacı (Binary Search Tree):
  - İkili arama ağacında bir düğüm en fazla iki tane çocuğa(derece) sahip olabilir ve alt/çocuk bağlantıları belirli bir sırada yapılır.
- Kodlama Ağacı (Coding Tree):
  - Bir kümedeki karakterlere kod ataması için kurulan ağaç şeklidir. Bu tür ağaçlarda kökten başlayıp yapraklara kadar olan yol üzerindeki bağlantı değerleri kodu verir.
- Sözlük Ağacı(Dictonary Tree):
  - Bir sözlükte bulunan sözcüklerin tutulması için kurulan ağaç şeklidir. Amaç arama işlemini en performanslı şekilde yapılması ve belleğin optimum kullanılmasıdır.
- Kümeleme Ağacı (Heap Tree):
  - Bir çeşit sıralama ağacıdır. Çocuk düğümler her zaman aile düğümlerinden daha küçük değerlere sahip olur.

İkili Ağaçlar

- Derecesi 2'dir.
- Sonlu düğümler kümesidir.
- Bu küme boş bir küme olabilir (empty tree).
- Boş değilse şu kurallara uyar.
  - Kök olarak adlandırılan özel bir düğüm vardır.
  - Her düğüm en fazla iki düğüme bağlıdır.
    - Left child : Bir düğümün sol işaretçisine bağlıdır.
    - Right child : Bir düğümün sağ işaretçisine bağlıdır.
  - Kök hariç her düğüm bir daldan gelmektedir.
  - Tüm düğümlerden yukarı doğru çıkıldıkça sonuçta köke ulaşılır.
- Bilgisayar bilimlerinde en yaygın kullanılan ağaçtır.

İkili Ağaçlar





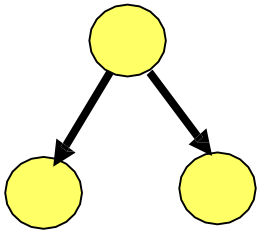
İki farklı ikili ağaç

İkili Ağaçlar

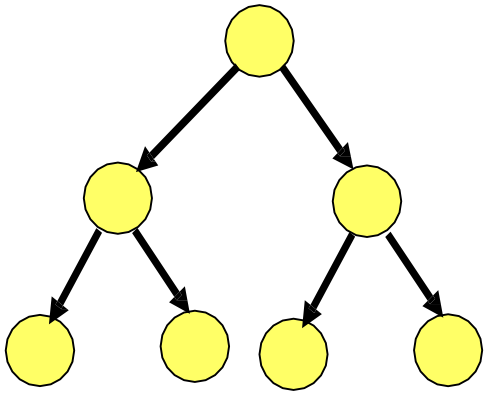
➢ İkili Ağaçların Özellikleri

- **Minimum Derinlik d: n=$2^{d+1} - 1$ (tam ikili ağaç)**

Derinlik 0: n =$2^{0+1} - 1$ =1 düğüm olabilir, n



Derinlik 1: n =$2^{2} - 1$ =3 düğüm olabilir, n



Derinlik 2: n =$2^{3} - 1$ =7 düğüm olabilir, n

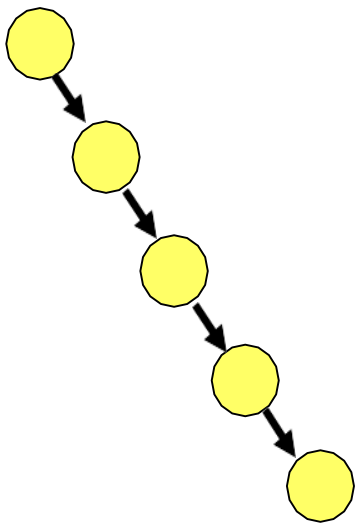
Herhangi bir d derinliğinde, n=?

Minimum derinlik: O(log N)

İkili Ağaçlar

İkili Ağaçların Özellikleri

- Maksimum Derinlik

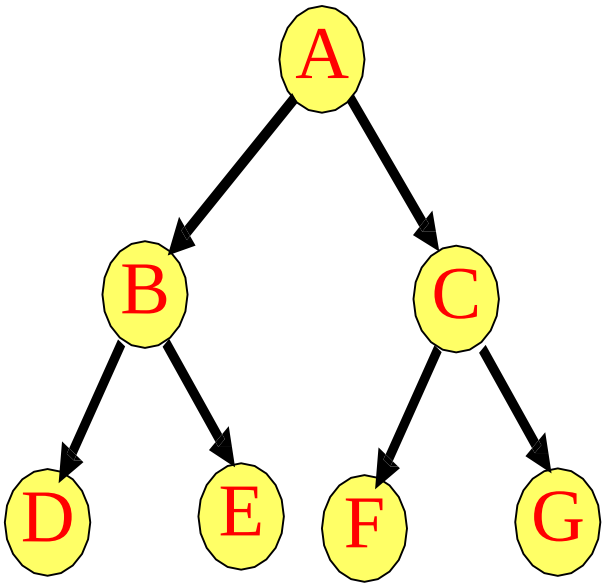


- Dengesiz Ağaç: Ağaç bir bağlantılı liste olursa!
- Maksimum derinlik = N

- Amaç: Arama gibi operasyonlarda bağlantılı listeden daha iyi performans sağlamak için derinliğin log N de tutulması gerekmektedir.

İkili Ağaçlar

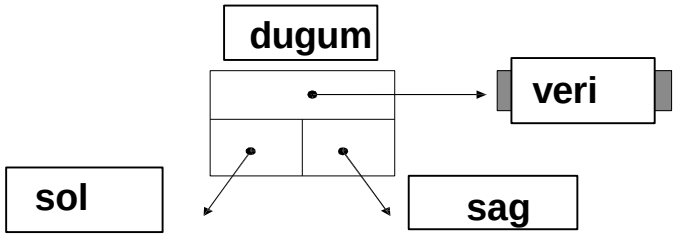
- İkili Ağaç örneği
  - Derecesi 2, derinliği 2 olan ağacı dizi veri yapısında tutmak için;

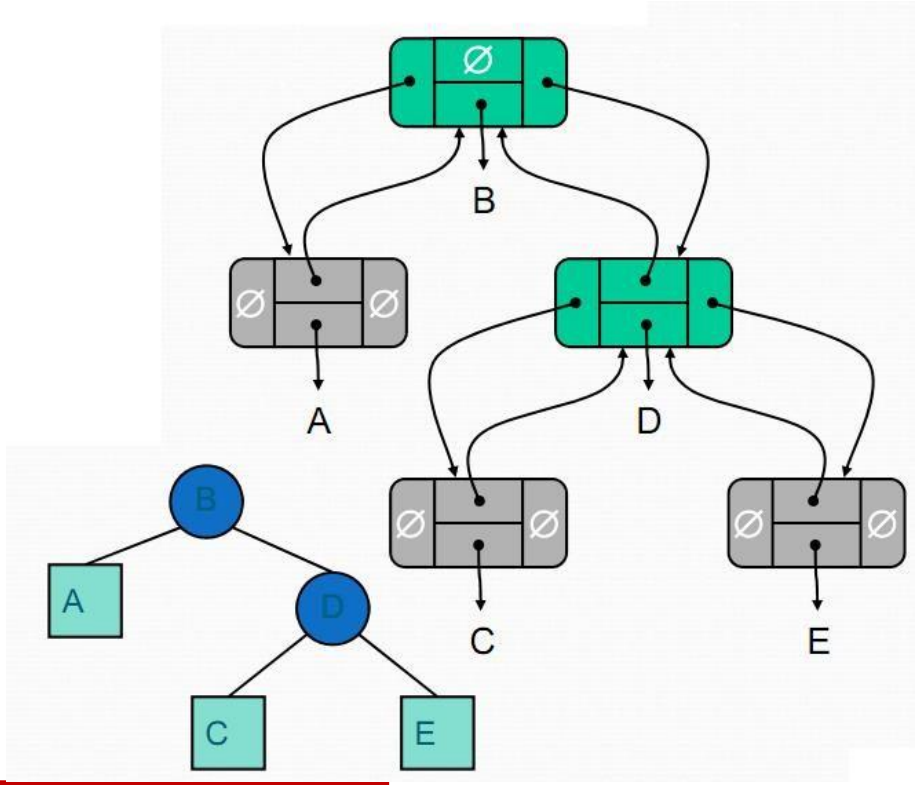


İkili ağaç için gereken dizi büyüklüğü $2^{d+1} - 1 = 7$ olur.

İkili Ağaçlar

➢ İkili Ağaç Bağlı Liste Yapısı



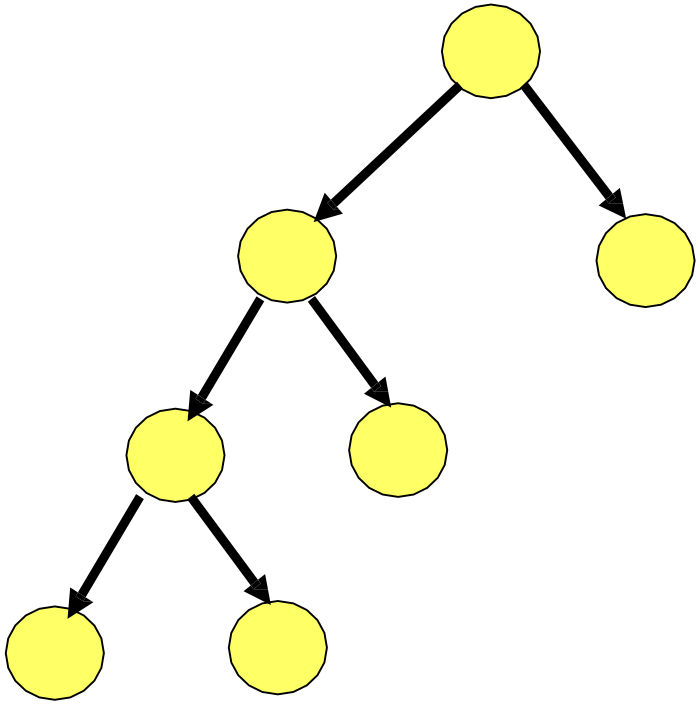


İkili Ağaçlar

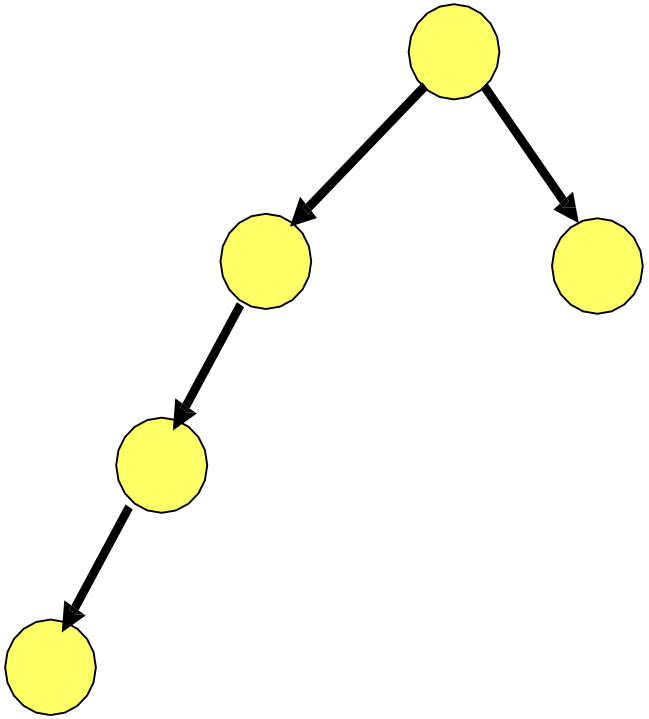
➢ Full Binary Tree (Proper Binary Tree)

- Yaprak olmayan düğümlerin tümünün iki çocuğu olan T ağacı proper (düzgün) binary tree olarak adlandırılır. Başka bir deyişle düğümleri 0 veya 2 çocuğa sahip ağaç türüdür.

Proper Tree



Improper Tree



İkili Ağaçlar

➢ Full Binary Tree (Proper Binary Tree)

- Bir full binary tree'de m tane yaprak varsa bu ağaçta toplam n=2m-1 düğüm vardır. Başka bir şekilde ifade edilirse,
- Eğer T ağacı boş ise, T yüksekliği 0 olan bir full binary ağaçtır.
- T ağacının yüksekliği h ise ve yüksekliği h'den küçük olan tüm node'lar iki child node'a sahipse, T full binary tree'dir.
- Full binary tree'de her node aynı yüksekliğe eşit sağ ve sol alt ağaçlara sahiptir. En iyi bilinen proper tree ağacıdır.

İkili Ağaçlar

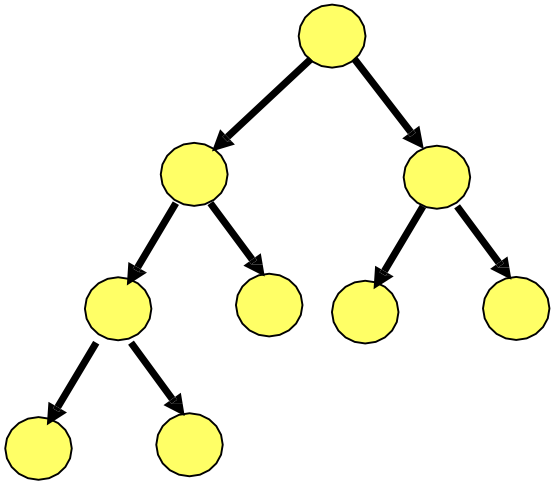
➢ Complete Binary Tree

- Full Binary Tree'de, yeni bir derinliğe soldan sağa doğru düğümler eklendiğinde oluşan ağaçlara Complete Binary Tree denilir.
- Böyle bir ağaçta bazı yapraklar diğerlerinden daha derindir. Bu nedenle Full Binary Tree olmayabilirler. En derin düzeyde düğümler olabildiğince soldadır.
- T, n yükseklikte Complete Binary Tree ise, tüm yaprak node'ları n veya n-1 derinliğindedir ve yeni bir derinliğe soldan sağa doğru ekleme başlanır.
- Her node iki tane child node'a sahip olmayabilir.

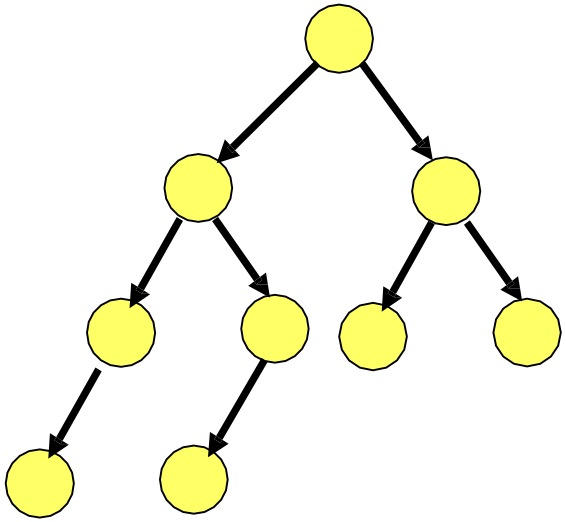
İkili Ağaçlar

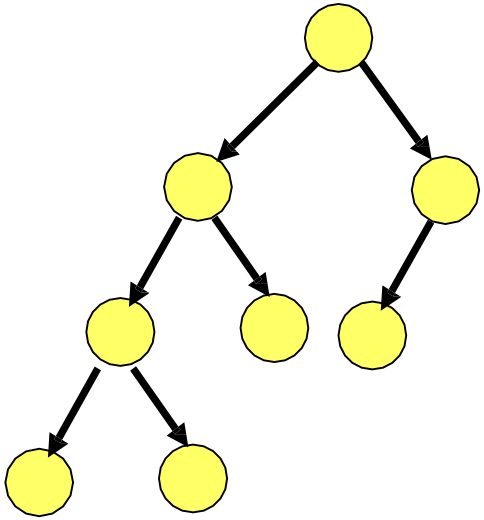
- Complete Binary Tree

Complete



Incomplete

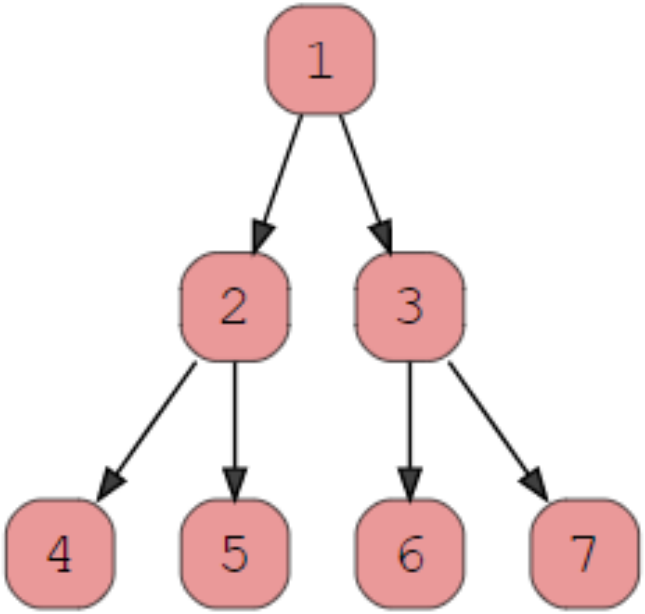




İkili Ağaçlar

➢ Perfect Binary Tree

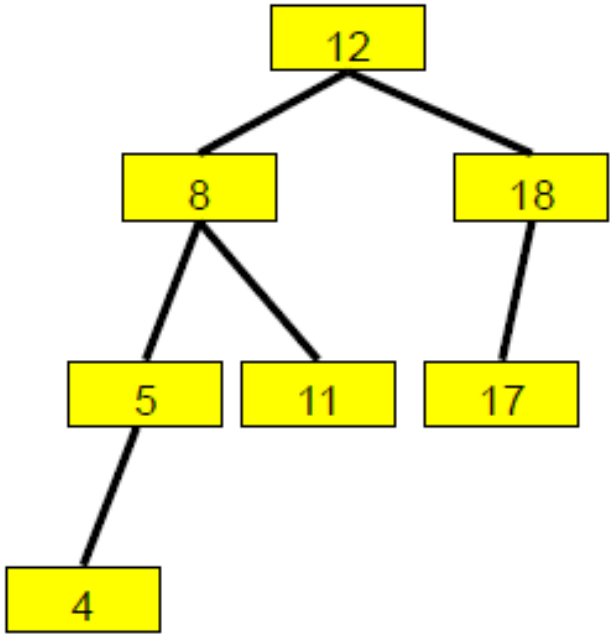
- Hem full binary tree hem de complete binary tree olmalıdır.
- Tüm iç düğümlerin iki çocuğu olmalı ve tüm yapraklar aynı derinliğe sahip olmalıdır.



İkili Ağaçlar

➢ Balanced (Dengeli) Binary Tree

- Yüksekliği ayarlanmış ağaçlardır.
- Bütün düğümler için sol alt ağacın yüksekliği ile sağ alt ağacın yüksekliği arasında en fazla bir fark varsa balanced binary tree olarak adlandırılır.
- Complete binary tree'ler aynı zamanda balanced binary tree'dir.

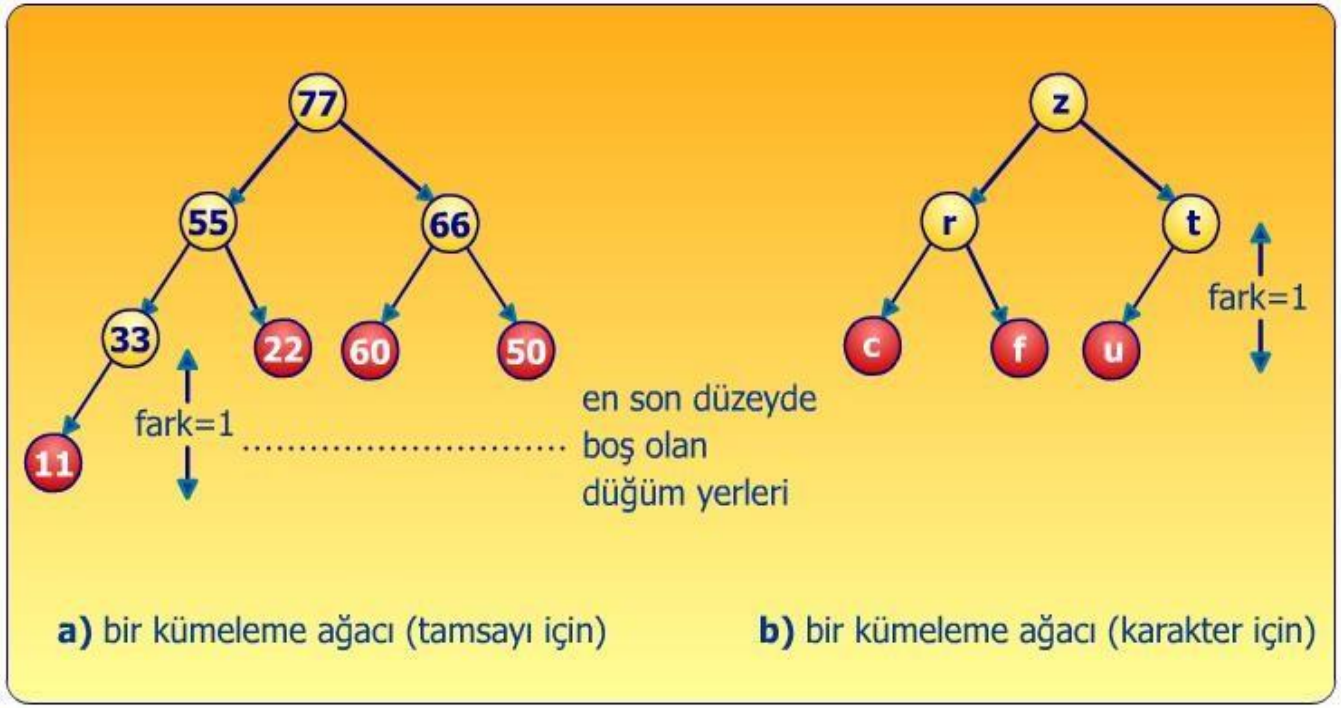


Heap Tree (Kümeleme Ağacı)

- Kümeleme ağacı, ağaç oluşturulurken değerleri daha büyük olan düğümlerin yukarıya, küçük olanların da aşağıya eklenmesine dayanan tamamlanmış bir ağaçtır.
- Böylece 0. düzeyde, kökte, en büyük değer bulunurken yaprakların olduğu k. düzeyde en küçük değerler olur. Yani büyükten küçüğe doğru bir kümeleme vardır.
- Tersi olarak küçükten büyüğe kümeleme de yapılabilir.
- Aksi belirtilmediği sürece kümeleme ağacı, sayısal veriler için büyükten küçüğe, sözcükler için alfabetik sıralamaya göre yapılır.
- Kümeleme ağacı bilgisayar biliminde bazı problemlerin çözümü için çok uygundur; hatta birebir uyuşmaktadır denilebilir. Üstelik, hem bellek alanı hem de yürütme zamanı açısından getirisi olur.

Heap Tree (Kümeleme Ağacı)

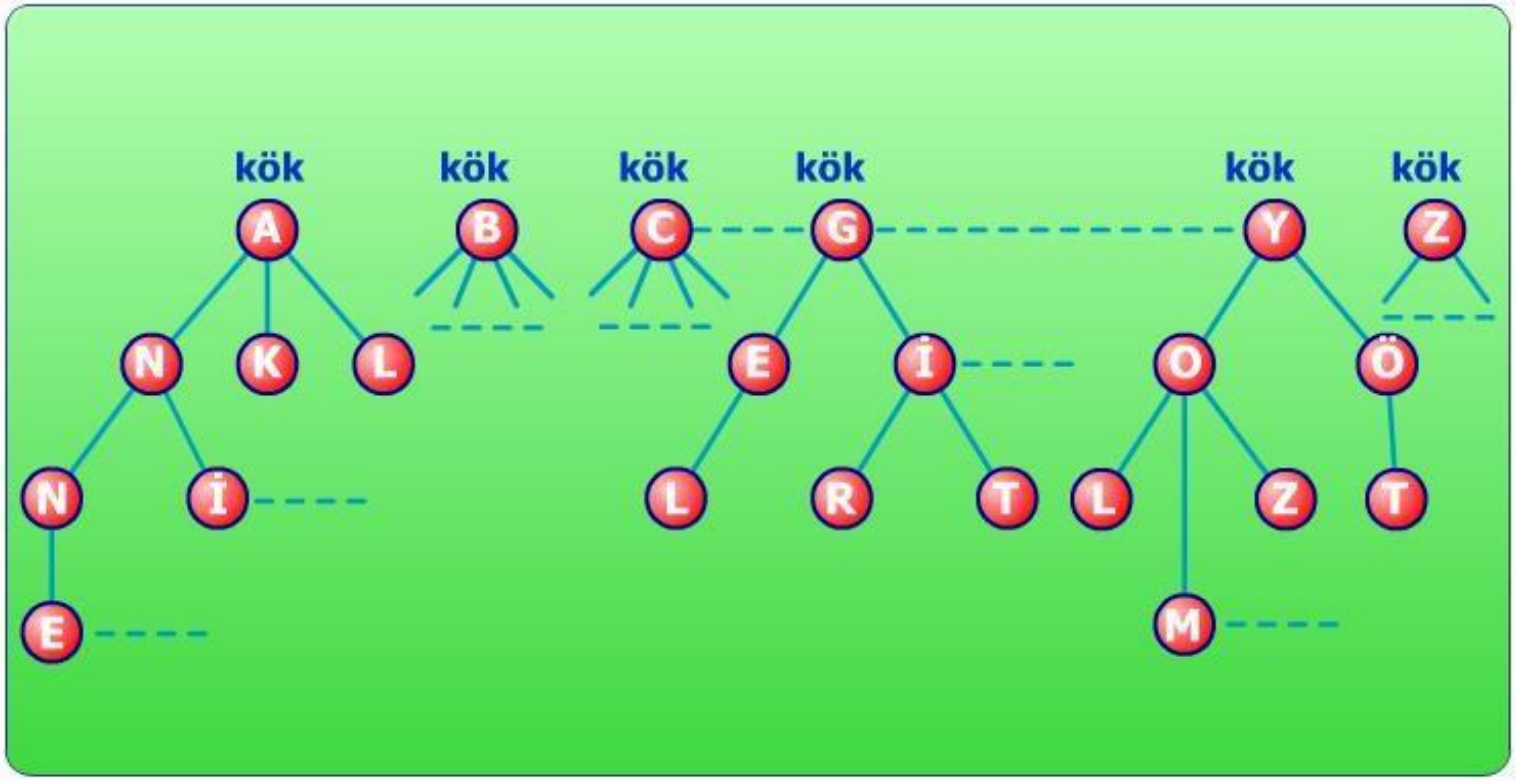
- Bir düğüm her zaman için çocuklarından daha büyük değere sahiptir.
- Yaprak düğümlerin herhangi ikisi arasındaki fark en fazla 1'dir.
- En son düzey hariç tüm düzeyler düğümlerle doludur.
- En son düzeyde oluşan boşluklar sadece sağ taraftadır.
- Sıralama işlemlerinde kullanılır.



Trie Ağacı/Sözlük Ağacı

- Sözlük ağacı, bir sözlükte bulunan sözcükleri tutmak ve hızlı arama yapabilmek amacıyla düşünülmüştür; bellek gereksinimi arttırmadan, belki de azaltarak, on binlerce, yüzbinlerce sözcük bulunan bir sözlükte 10-15 çevrim yapılarak aranan sözcüğün bulunması veya belirli bir karakter kadar uyuşanının bulunması için kullanılmaktadır.
- Sözlük ağacı, sıkıştırma, kodlama gibi sözlük kurulmasına dayalı algoritmalarda ve bir dilin sözlüğünü oluşturmada kullanılmaktadır.

Trie Ağacı/Sözlük Ağacı

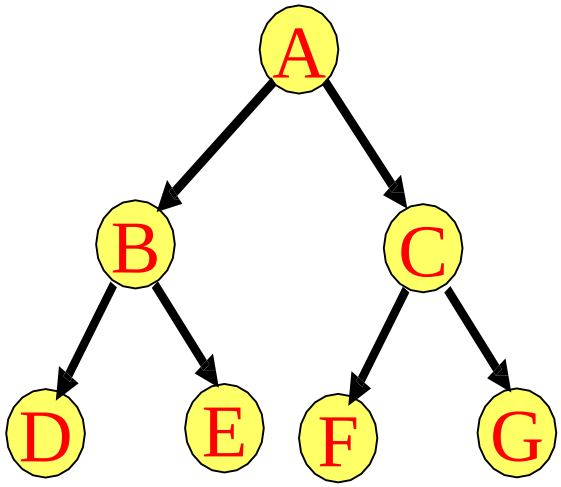


İkili Ağaçlarda Geçiş İşlemleri

- Ağaçlar üzerinde veri arama, listeleme, düğüm ekleme veya silme işlemleri için gezinmek gerekir.
- Geçiş (traverse), ağaç üzerindeki düğümlere uğrayarak gerçekleştirilir.
- Ağaçlar düğümleri doğrusal olmadığından gezinme için farklı algoritmalar uygulanır.
- İkili ağaç üzerinde gezinme için 3 farklı yaklaşım kullanılır:
  - Preorder(Önce düğüm)
  - Inorder(Ortada düğüm)
  - Postorder(Sonra düğüm)

İkili Ağaçlarda Geçiş İşlemleri

- Preorder (Önce düğüm – NLR / NRL)
  - Düğüme uğra
  - Sol alt ağacı preorder olarak dolaş.
  - Sağ alt ağacı preorder olarak dolaş.
- Önce düğüm yaklaşımı; ağacın kopyasını elde etme, düğümleri sayma, yaprakları sayma, matematiksel ifadeler için prefix gösterim elde etmek için kullanılır.

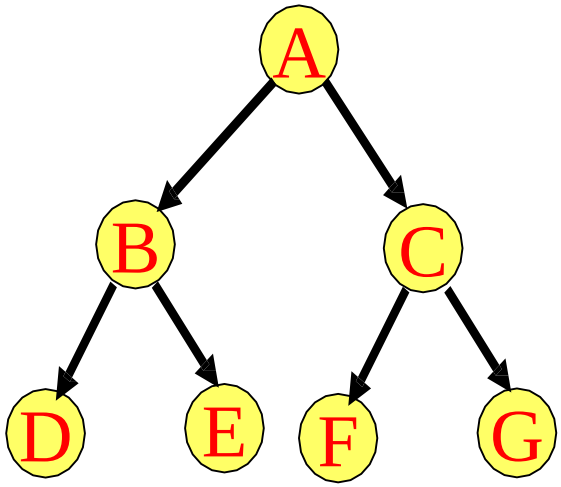


NLR: A, B, D, E, C, F, G

NRL: A, C, G, F, B, E, D

İkili Ağaçlarda Geçiş İşlemleri

- Inorder (Ortada Düğüm – LNR / RNL)
  - Sol alt ağacı inorder'a göre dolaş
  - Düğüme uğra
  - Sağ alt ağacı inorder'a göre dolaş.
- Tasarımcının tercihine göre sağ alt ağaç dolaşıldıktan sonra sol alt ağaç dolaşılabilir.
- Ortada düğüm yaklaşımı; ikili arama ağacında sıralama yapmak için kullanılır.

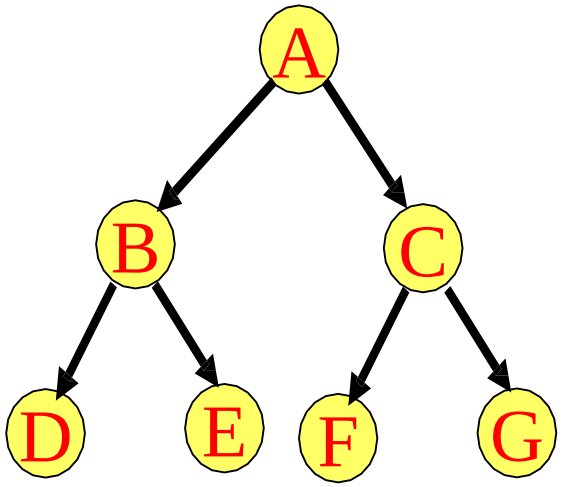


LNR: D, B, E, A, F, C, G

RNL: G, C, F, A, E, B, D

İkili Ağaçlarda Geçiş İşlemleri

- Postorder (Sonra Düğüm – LRN / RLN)
  - Sol alt ağacı postorder'a göre dolaş
  - Sağ alt ağacı postorder'a göre dolaş
  - Düğüme uğra
- Sonra düğüm yaklaşımı; ikili arama ağacında silme işleminde, fonksiyonel programlama dili derleyicilerinde, hesap makinesi yazılımlarında ve postfix matematiksel ifadelerin oluşturulmasında kullanılır.

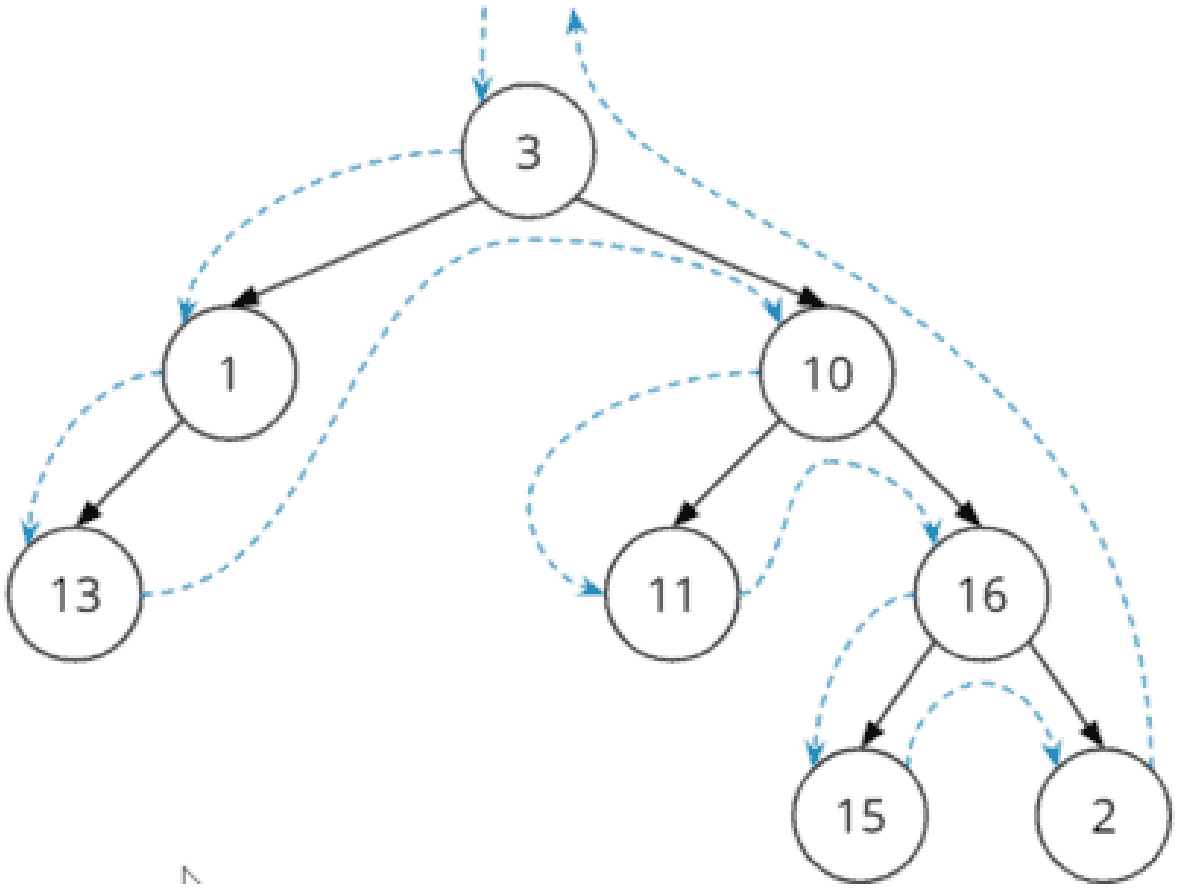


LRN: D, E, B, F, G, C, A

RLN: G, F, C, E, D, B, A

İkili Ağaçlarda Geçiş İşlemleri

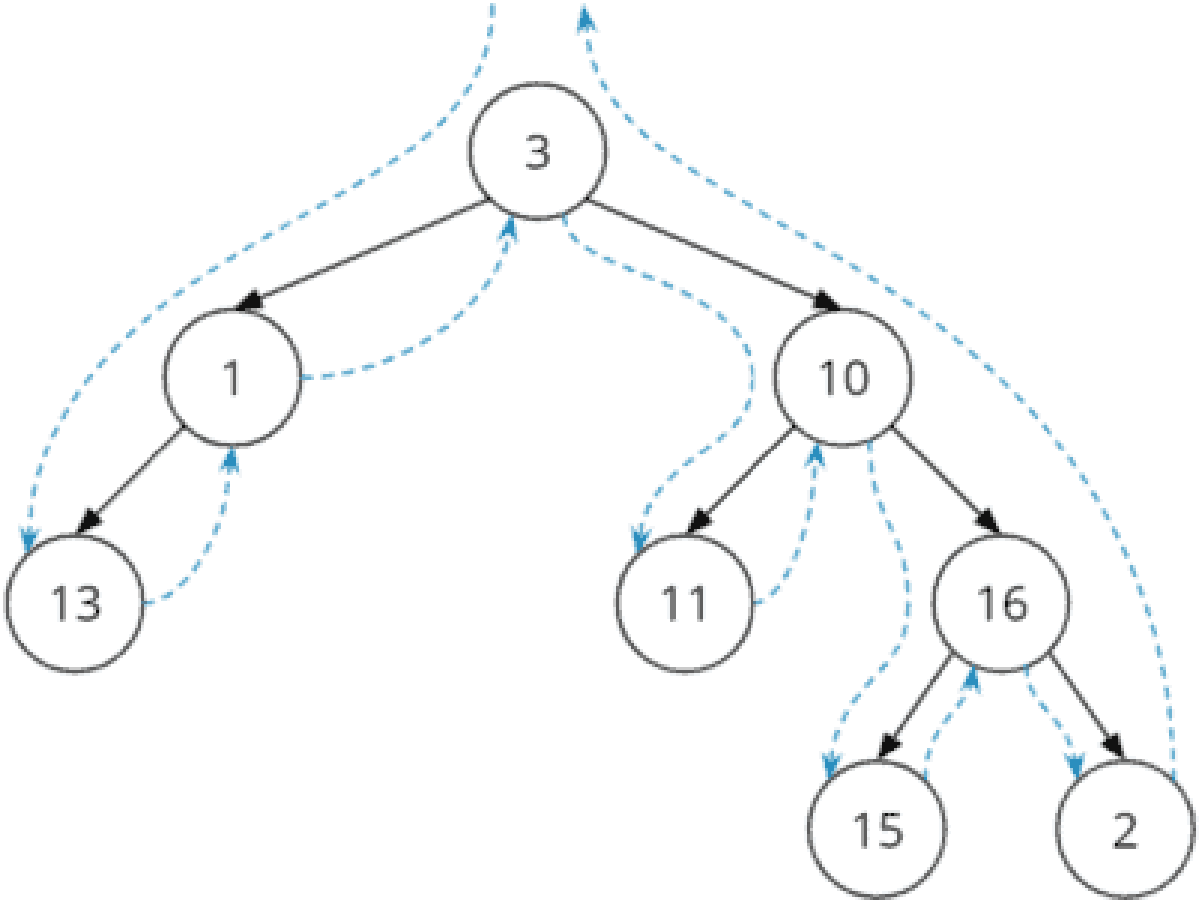
- Preorder Geçişi(NLR)



3→1→13→10→11→16→15→2

İkili Ağaçlarda Geçiş İşlemleri

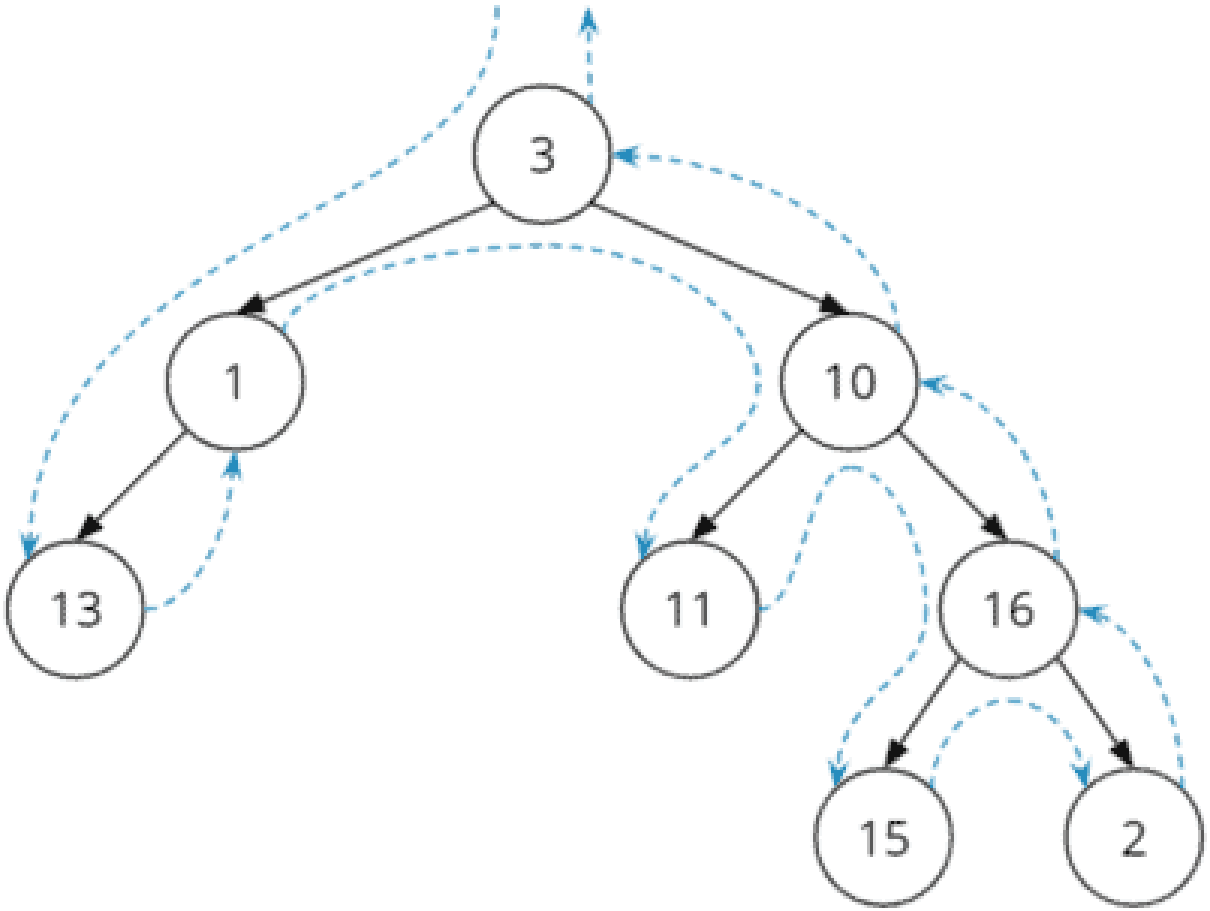
- Inorder Geçişi(LNR)



13→1→3→11→10→15→16→2

İkili Ağaçlarda Geçiş İşlemleri

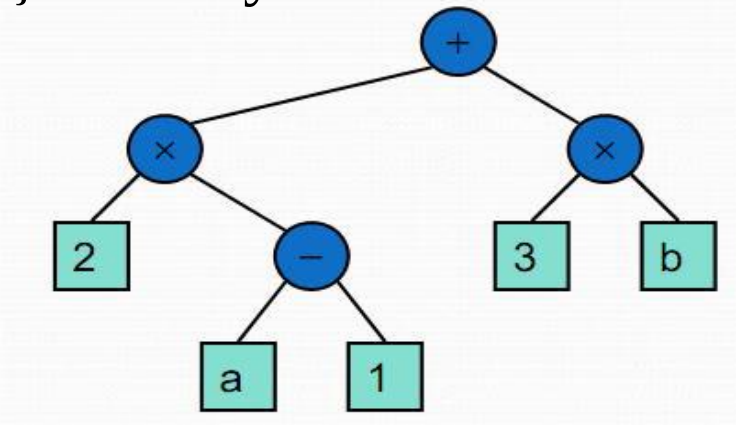
- Postorder Geçişi(LRN)



13→1→11→15→2→16→10→3

Bağıntı Ağacı (Expression Tree)

- Bağıntı(İfade) ağaçları matematiksel bir bağıntının çözümlenmesi için kullanılır.
- Bağıntı ağacı ile işlem önceliği ve parantez kullanımı kolayca ifade edilebilir.
- Bağıntı ağacında;
  - Yapraklar değişkeni veya sabit değeri,
  - Kök veya ara düğümler aritmetik veya mantıksal operatörleri gösterir.
- Birçok derleyicide kullanılır. Parantez gereksinimi yoktur.



$$(2 \times (a - 1) + (3 \times b))$$

İfade Ağacı (Expression Tree)

➢ İfade Ağaçlarında Geçiş

- infix: Klasik matematikte olduğu gibi değişkenler operatörün etrafına yayılır. Ağacın çözümlenmesi ortada kök yaklaşımı ile gerçekleştirilir.

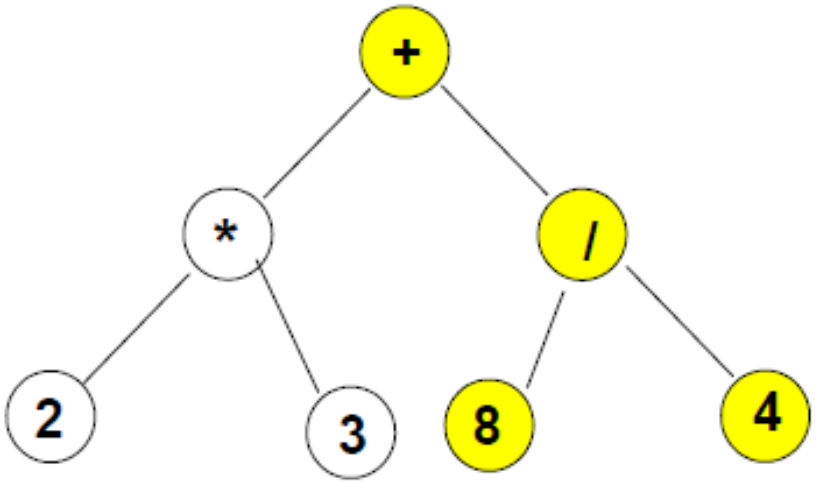
  ((2 * 3) + (8 / 4))

- prefix

  + * 2 3 / 8 4

- postfix

  2 3 * 8 4 / +



Örnek Uygulamalar

1) İkili arama ağacını bağlı liste ile oluşturup; ağaca eleman ekleme, preorder, inorder ve postorder dolaşma fonksiyonlarını yazınız.
2) Bir "postfix" ifadesinden ifade ağacını oluşturmak için algoritma aşağıdadır. Buna göre "ABC*+D/" postfix ifadesi için ifade ağacı oluşturan ve ortada kök ile dolaşan kodu yazınız.

Algoritma:

1) İfadeyi soldan sağa tarayın.

2) Bir "operand" bulunursa. Ardından bu "operand" için bir düğüm oluşturun ve onu yığına itin.

3) Bir "operator" bulunursa. Yığından iki düğümü alın (pop) ve "operator" ü kök düğüm olarak ve iki öğeyi sol ve sağ alt öğe olarak tutan bir düğüm yapın. Yeni oluşturulan düğümü yığına itin.

4) Postfix ifademizin sonuna gelene kadar yukarıdaki iki adımı tekrar etmeye devam edin.

5) Yığıtta geride kalan düğüm, ifade ağacının başını temsil eder.